Object Oriented Programming With 'C++'
B.C.A-I (Sem-II) Year: 2013-2014

Introduction:

Programming languages are used to create programs i.e. software. And they are classified as:

1) Procedure oriented language.
2) Object oriented language.

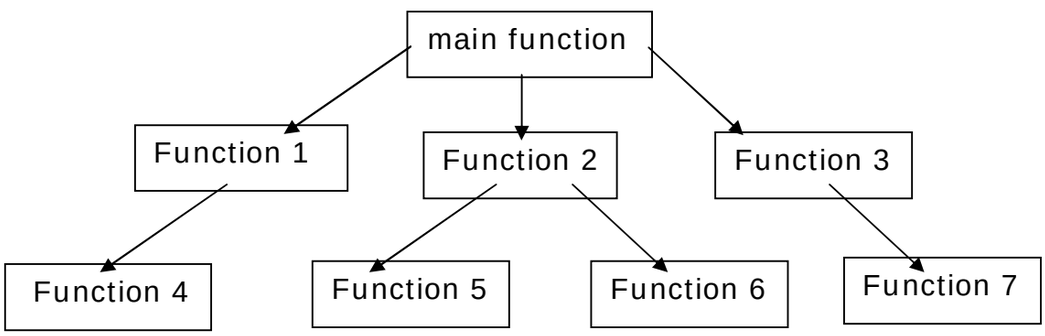
Procedure Oriented Programming (POP):

The POP is also called as functional programming because the strength of POP languages is function.

High level languages such as COBOL, FORTRON, and C etc. are commonly known as POP languages.

- The programmer organizes data and functions whenever required.
- In POP the focus is on function rather than data.
- Basically POP consist of several functions & each function performs its own task.
- In multifunction program many important data is placed at global section such that all function can easily access this data.

The POP structure is shown as fallows:



e.g. Consider the payroll system which is used to display or generate employee pay slip, for that we consider three functions accept( ), calculate( ) and display( ).

The working of all these functions are as fallows:

1) accept( ) = This function is used to accept all the details of employee such as emp_no, emp_name, emp_dob, emp_add, basic_salary etc.
2) calculate( )= This function is used to calculate Net payment or gross salary of employee by considering HRA, TA, DA etc.
3) display( )= This function is used to display the pay slip details of employee.

Advantages or Characteristics of POP languages:

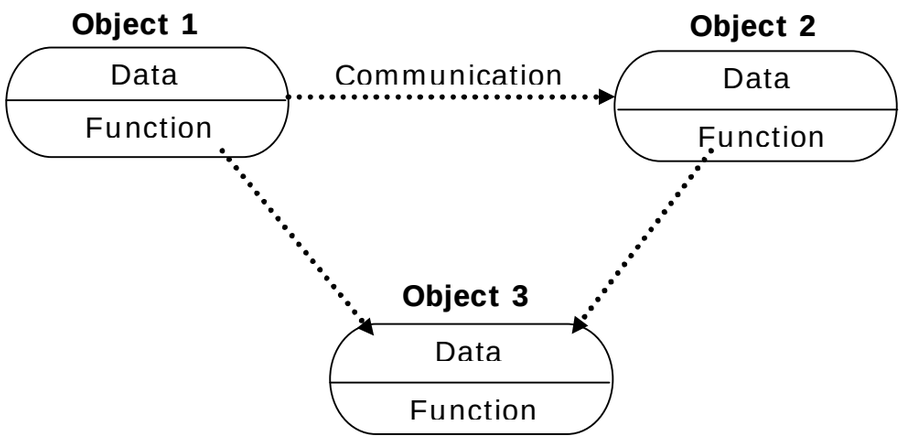
1) In case of POP the focus is on Function rather than data.
2) It fallows top-down approach.
3) Program consists of different functions.
4) Most function share global data.
5) Communication is done between functions i.e. one function can send data to another function.
6) Data is not hidden i.e. data is easily accessed outside the world.

Object Oriented Programming (OOP):

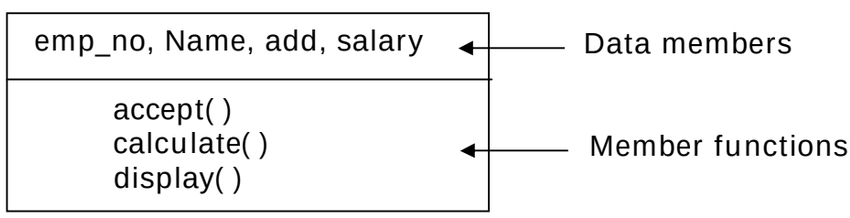
- The OOP contains the concepts of POP and also added some extra features such as Class, Object, Polymorphism, Data abstraction, Inheritance, Message passing etc. The languages such as C++, Java, C# etc. are commonly known as OOP languages because they supports for OOP's concepts.
- OOP allows decomposition of a problem into number of entities called as Object and then builds data and function around this object.
- In OOP the focus is on Data rather than function.
- Once the data which is to be used is decided first and then different functions are implemented according to the data. Thus, OOP follows bottom-up approach.
- The OOP allows data hidden therefore data cannot be accessed outside the world.
- In OOP the objects sends messages to one another i.e. communication is happen between objects.

- The OOP approach eliminates these weaknesses of procedure-oriented programming approach by combining the data and the function that work on it into a single unit called class.
- In case of OOP the data is secured from the outside world because data can be accessed by the associated function only (Member functions) and not by non member functions.
- OOP is implemented on the real-world entities called objects that have some characteristics and behavior.
- An object comprises both data and function. The data describes the state and function describes the behavior of the object.

Following figure shows structure of OOP:



e.g. Consider following payroll system which generates pay sleep of employee.



In above example: emp_no, Name, add and salary are the data of employee class is decided first and then different functions such as accept( ), calculate( ) and display( ) are defined thus, OOP follows bottom-up approach.

Advantages or Characteristics of OOP languages:

1) In case of OOP, the focus is on data rather than function.
2) It follows bottom-up approach.
3) Program consists of different objects.
4) Communication is done between objects i.e. one object can send data to another object.
5) Data is hidden i.e. it can not accessed outside the world.
6) New data & functions can easily be added whenever required.

Distinguish between POP and OOP:

POP	OOP
1) The focus is on function rather than data.	1) The focus is on data rather than function.
2) It follows top-down approach.	2) It follows bottom-up approach
3) Program consists of different Functions.	3) Program consists of different objects.
4) Communication is done between functions.	4) Communication is done between objects
5) Data is not hidden.	5) Data is hidden.
6) New data & functions cannot be added.	6) New data & functions can be added whenever required.

7) COBOL, FORTORN, C etc are the examples of POP language.	7) C++ is the example of OOP language.
8) This is a good programming approach for real time software design	8) This is good for scientific application development
9) It does not supports for the features of OOP's	9) All features of POP may be present in OOP

* **Note**: OOP languages are those languages which supports for OOP concepts such as Class, Object, Data encapsulation, Data abstraction, Message passing, Polymorphism, Inheritance etc.

OOP concepts:

There are several OOP concepts which are as fallows:

1) Class:

Class is user defined data type which is collection of data member and member functions. Class can be declared or defined by using keyword 'class' followed by 'class_name' which is an identifier.

Syntax to declared or defined class:

```
class      class_name
{
   data_type        datamem1;          ←——— Data members
   data_type        datamem2;
        ⋮                ⋮
   data_type        datamemN;

   return_type     mem_fun1( );
   return_type     mem_fun2( );        ←——— Member functions
        ⋮                ⋮
   return_type     mem_funN( );
} ;
```

- By default all the data of class are private therefore this data can not be accessed out side the world.
- Multiple objects can be created for the single class whenever required and each object hold individual record.
- A class provides the blueprints for objects.
- When you define a class, you define a blueprint for a data type. This doesn't actually define any data, but it does define what the class name means i.e. what an object of the class will consist of and what operations can be performed on such an object.

e.g. Following example shows the definition of employee class:

```
class          emp
{
        int       emp_no;
        char      name[20], add[20];     ←——— Data members (Attributes)
        float     salary;

        void      get( );
        void      show( );               ←——— Member functions (Operations)
} ;
```

Note: *The functionality of class and structure is same but ,*

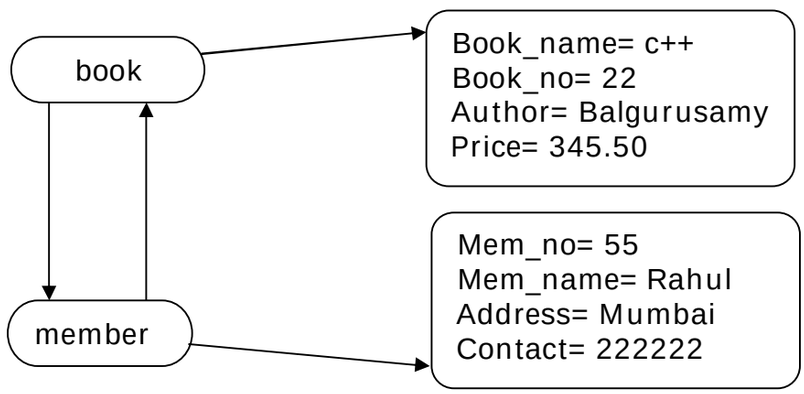
1) By default all the data of structure is public where as all the data of class is private.

2) Generally structure is only collection of data members where as class is collection of data members as well as member functions.

2) Object:

- An Object is an instance or variable of class.
- An object can be thought of as an entity which can represent person, place, animal etc.
- Object is such run time entity which holds entire data and functions of class.
- When program is executed, the objects interact with each other by sending messages to one another. That is due to object message passing is also done.
- There are three properties related with objects:
  - Identity: This property of an object that distinguishes it from other objects
  - State: This property describes the data stored in the object
  - Behavior: This property describes the methods in the object's interface by which the object can be used
- Multiple objects can be created for the single class whenever required and each object hold individual record.

e.g. Consider the library system which has different objects such as 'book' and 'member'. The communication between 'book' and 'member' object is shown as fallows:



Syntax to create object:

```
Class_name        object_name;
```

OR

```
class   Class_name        object_name;
```

Here; 'Class_name' is name of class which is an identifier.
'object_name' is name of object which is also identifier.

e.g. Following statements declare two objects of class 'Box':

```
Box     Box1;     / / Declare Box1 of type Box i.e. Box1 is object
Box     Box2;     / / Declare Box2 of type Box i.e. Box2 is object
```

Both of the objects Box1 and Box2 will have their own copy of data members

Purpose to create object:

- To store entire data of class whether it is data members or member functions
- To access public data of class in any non-member function

Accessing the Data Members:

The public data members of objects of a class can be accessed using the direct member access operator (.) Let us try following example to make the things clear:

```
class   Box
{
  public:
    double    length;        / / Length of a box
    double    breadth;        / / Breadth of a box
    double    height;        / / Height of a box
};
```

Object of above class can be created as:

```
Box    b1, b2;
```

And data members are accessed and initialized as:

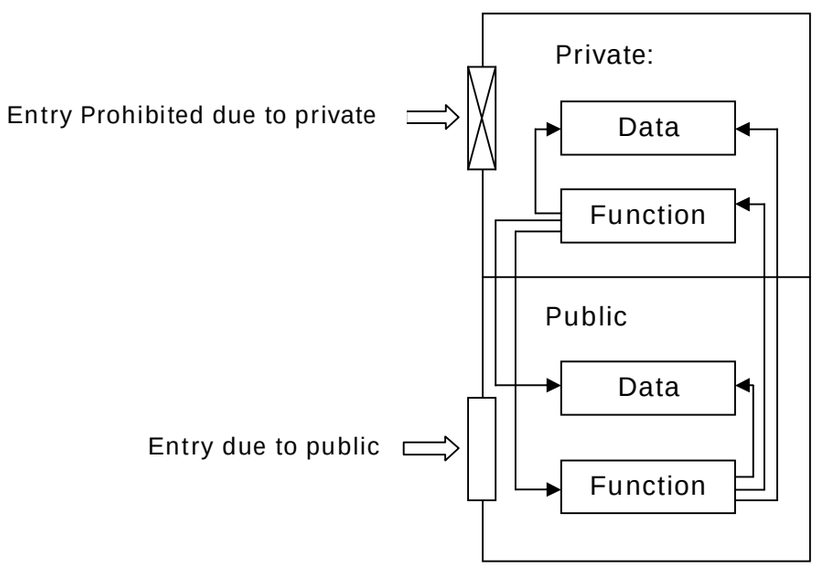
```
b1.length = 45.2;
b1.breadth = 78.52;
b1.height = 10.50;

b2.length = 75.2;
b2.breadth = 70.52;
b2.height = 20.50;
```

Here, 'b1' object hold one record and 'b2' object hold another record.

3) Data Encapsulation:

- The binding of data and functions into single unit is called "Data encapsulation". And that keeps both data and functions safe from outside the world and misuse.
- Data encapsulation lead to the important OOP concept of data hiding and is accomplished by class. Because, by default all the data of class is 'private' therefore this data is only accessed by functions present in the class (Member Functions) and not accessed by outside the world (Non-Member functions).
- C++ supports the properties of encapsulation and data hiding through the creation of user-defined types, called 'classes'
- A benefit of encapsulation is that it can reduce system complexity, and thus increases robustness, by allowing the developer to limit the interdependencies between software components.
- This feature supports the data security in OOP.
- The data can be modified or accessed by the associated member functions.

Following figure shows Data encapsulation concept:



4) Data Abstraction:

- Data abstraction refers to, providing only essential information to the outside word and hiding their background details.
- This is closely related to encapsulation because abstraction is implemented by encapsulation.
- In C++, classes provide great level of data abstraction. They provide sufficient public methods to the outside world to play with the functionality of the object and to manipulate object data. Without actually knowing how class has been implemented internally.
- If we have to give accessibility to some data of the class then it has to be placed in public section and those we have to make hide from outside world then it is placed in private section of class. Thus, Classes makes data abstraction with the help of access specifier or visibility mode.
- E.g.

  Let's take one real life example of a TV. Which you can turn on and off, change the channel, adjust the volume, and add external components such as speakers, VCRs, and DVD players.

  But you do not know its internal details such as,
  - you do not know how it receives signals over the air or through a cable
  - How it translates them, and finally displays them on the screen.

Thus we can say, a television clearly separates its internal implementation from its external interface and you can play with its interfaces like the power button, channel changer, and volume control without having zero knowledge of its internals

5) Polymorphism:

- The word polymorphism is made up of two Greek words “poly” and “morphism”.
- “Poly” means many and “morphism” means forms, so polymorphism means many-forms.
- In OOP, we implement many functions with same name but these functions are differ from each other according to their signature. (Signature refers to type of argument and number of argument accepts by the functions)
- In OOP, polymorphism means “one name” named for multiple functions that have different behaviors.
- E.g. Consider following prototype of functions:

```
int     add( );
void    add(int);
int     add(int, int);
```

  In above example, many functions having same name but all these differ from each other by signatures therefore here polymorphism occurs.

- Following figure shows polymorphism with its types:

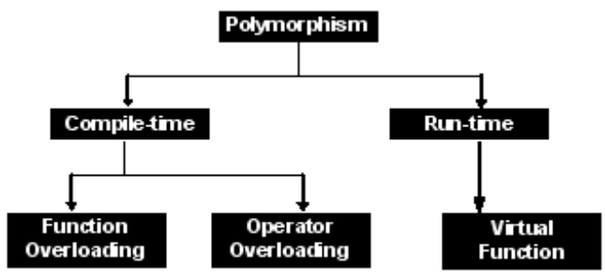


Fig: 11.1: Polymorphism in C++

Basically polymorphism having two types:

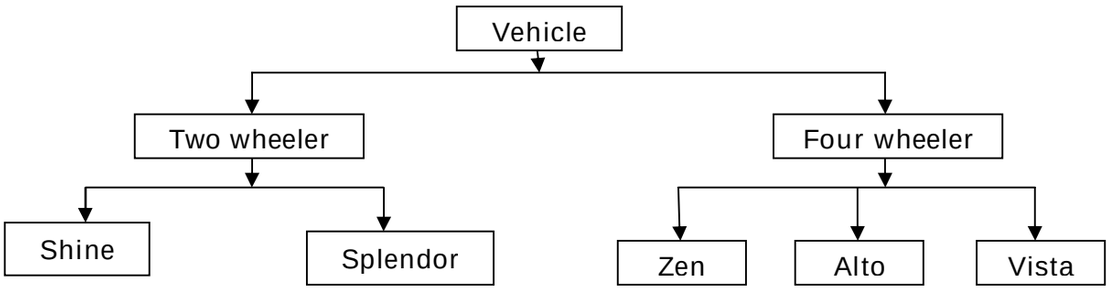
1) Compile Time polymorphism (Static linkage/Static Binding/Early Binding):
   Definition: Choosing the functions at compile time of program is called as “Compile time polymorphism or Static linkage/Static Binding/Early Binding”
   Compile time polymorphism is achieved by ‘function overloading’ and ‘operator overloading’

2) Run Time polymorphism (Dynamic linkage/Dynamic Binding/Late Binding):
   Definition: Choosing the functions at run time of program is called as “run time polymorphism or dynamic linkage/dynamic Binding/late binding”
   Runtime polymorphism is achieved by ‘Virtual Functions”

6) Inheritance:

- Inheritance is a one of the most important feature of OOP, where a class can inherits data members and member function (methods) of existing class.
- Inheritance is nothing but the process of deriving (Creating) new classes from existing class. The new class is called as ‘Derived class or sub class or child class’ where as existing class is called as ‘Base class or Super class or parent class’.
- In inheritance, any number of classes can be linked with one another.
- Inheritance supports for “Is-A” relationship.
- When creating a class, instead of writing completely new data members and member functions, the programmer can select that the new class should inherit the members of an existing class.
- While creating new class from existing one, the derived class accesses the data from base class and also add its own features into existing class without modifying base class. Thus, existing class is reused in derived class.
- The main concept behind inheritance is “Reusability” of code is possible. i.e. we use existing program again and again with addition of extra features.
- Inheritance reduces software developing time
- Inheritance reduces software developing cost also.

- Following figure shows concept of inheritance:



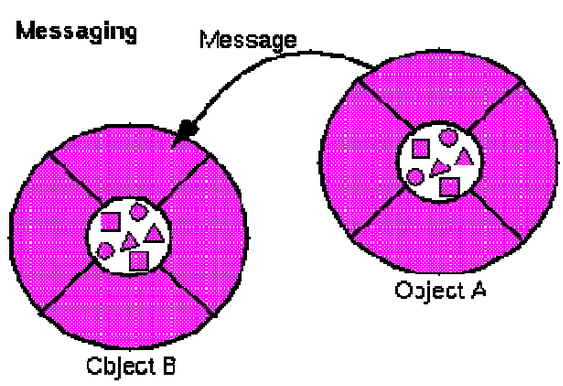
In above figure, 'Vehicle' is super base class from which two classes are derived 'Two wheeler' and 'Four wheeler'. Also, 'Shine' and 'Splendor' are derived classes from 'Two wheeler'. And 'Zen', 'Alto' and 'Vista' are derived classes from 'Four wheeler' base class.

7) Message Passing:

- The act of communicating with an object to get something done is called as "Message Passing"
- In message passing sending and receiving of information is done by the objects. So this helps in building real life systems. Following are the basic steps in message passing.
    - Creating classes that define objects and its behavior.
    - Creating objects from class definitions
    - Establishing communication among objects

- In OOPs, Message Passing involves specifying the name of objects, the name of the function, and the information to be sent.
- Objects, which are usually instances of classes, are used to interact with one another to design strong applications.
- In OOP, each object is capable of **receiving messages**, processing data, and **sending messages** to other objects and can be viewed as an independent 'unit' with a distinct role or responsibility.
- Objects react when they receive messages by applying methods on themselves.
- A message is a request to an object to invoke one of its methods; in other words, a message for an object is a simply a call to one of its methods through the object.
- When objects communicate with one another, we say that they send and receive messages.
- E.g. X.getdata(a,b);

Here, we are passing a message getdata()to the objects 'X' with parameters 'a' and 'b'. Following figure shows messaging between object A and Object B:



Introduction to C++ language

History of C++ language:

- C++ language was developed by 'Bjarne Stroustrup' at AT & T's Bells laboratory in USA in year 1979.
- The first C with Classes compiler was called 'Cfront', which was derived from a C compiler called CPre.
- Cfront was written mostly in C with Classes, making it a self-hosting compiler (a compiler that can compile itself)
- In 1983, the name of the language was changed from C with Classes to C++.
- He combines two programming languages to develop more powerful language C++
- He combines 'C' language and 'simula 67' language to develop C++.
- 'Simula67' language supports for several OOP concepts where as 'C' language is POP language. That's why C++ language supports for OOP and also POP. And hence it is multi paradigm language.
- 'C++' is superset of 'C' language i.e. every program in 'C' language is easily executed by C++ compiler.

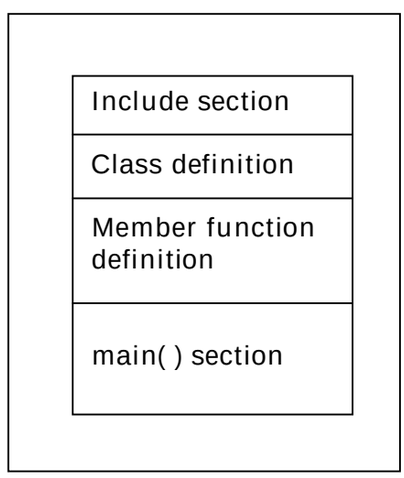
Application (Use) of C++ language:

- We know that, C++ is OOP's language which is used to develop large or complex programs (software's) such as editors, compilers, communication systems, database systems etc.
- C++ used in Computer Aided Design system.
- All major applications of Adobe Systems are developed in C++ such as Photoshop, ImageReady, Illustrator, Acrobat etc.
- All Unix-based server applications are completely coded In C++ language.
- MySQL Server (about 250,000 lines of C++) almost all coded in C++.
- C++ is mostly used to develop real life problems and use to develop application programs also.

Structure of C++ program:

- A typical every C++ program would contains four sections viz:
  a) Link section (Include File)
  b) Class declaration or class definition
  c) Member function definition
  d) main( ) function section

Following figure shows structure of C++ program:



Input and Output operations in C++:

In C++, the console based input and output operations are done by using 'cin' and 'cout' objects respectively.

- In C++ streams classes are of two types for inputting and outputting :
  - Input Stream.
  - Output Stream.

* Input Stream:

- The input stream handles all input operations or reading operations with input devices like keyboard, mouse etc.
- "cin" is predefined object of "istream class" and it is used for all console input operations.
- Syntax to use 'cin' object:

  cin>> input stream;

  Here,

'input stream' contains any variable, array or pointer etc.

>> is extraction operator.

E.g. int x,y,z;
cin>>x>>y>>z;

Here, Three integer values are input from keyboard and stored in variable 'x', 'y' and 'z'

* Output Stream:

- The output stream handles all output operations or writing operations with output devices like screen, printer, disk etc.
- "cout" is predefined object of "ostream class" and it is used for all console output operations.
- Syntax to use 'cout' object:

Cout<< output stream;

Here,

'output stream' contains any variable, array or pointer and also any message to display onto console etc.

<< is insertion operator.

E.g. 1) int x= 10;
Cout<<x;

Here, Value of variable 'x' is displayed onto console.

2) cout<< "Good morning";

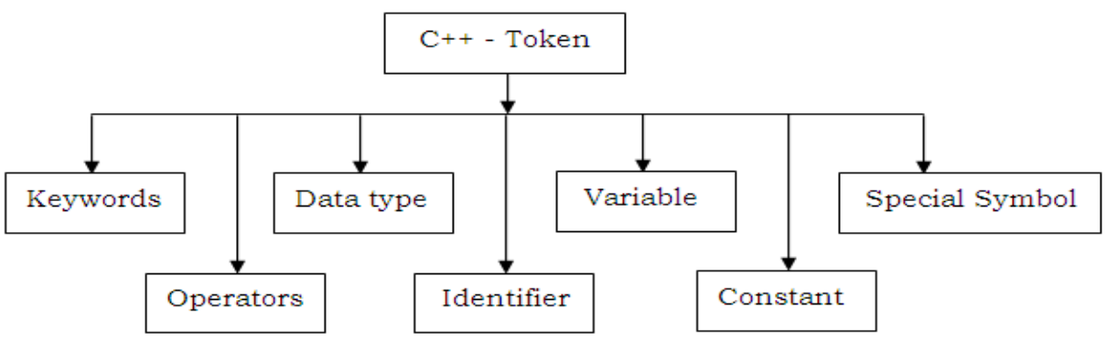
Here, Good morning is displayed onto console.

Note that: To use 'cin' and 'cout' object in program, we have to include "iostream.h" header file.

Tokens in C++:

Token is nothing but smallest individual unit of C++ program.

Following Figure shows C++ tokens:



(All the tokens of C++ are same as C language also C++ added some extra tokens)

Data Types in C++ language:

Data: " Data is nothing but collection of raw information or unprocessed information that we provide for the computer for processing"

e.g. numbers, string, alphanumeric etc.

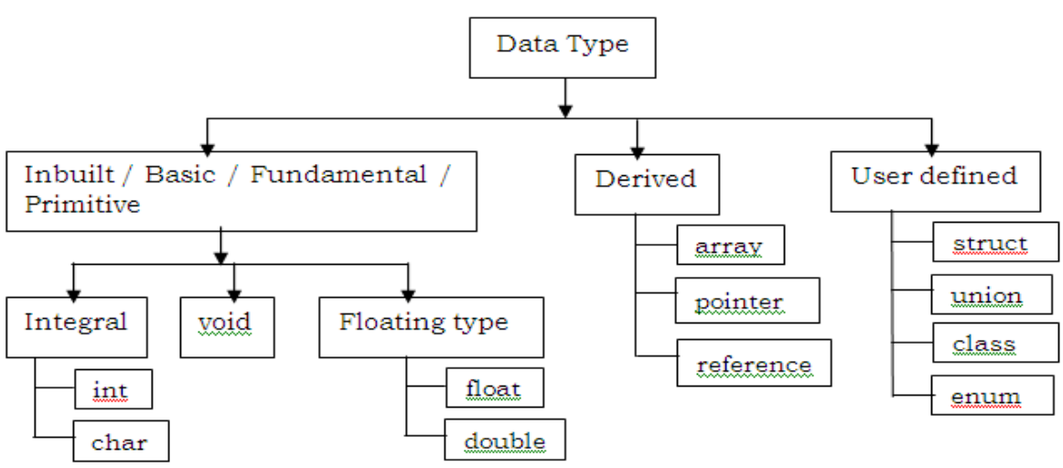
Data Type:

Concept: When we give data for processing to the computer at that time compiler does not know which type of input data is.

Generally, Data types are used to tell the compiler which type of input data is.

Definition: "Type of Data is called as Data Type"

Following Figure shows data types in C++ language:



(Instruction: Explain all above data types using 'C' language notes)

Difference between ‘C’ and ‘C++’:

‘C’ language	‘C++’ language
1) ‘C’ follows the procedural programming paradigm.	1)C++ is a multi-paradigm language(procedural as well as object oriented)
2) In case of C, importance is given to the steps or procedure of the program	2) C++ focuses on the data rather than the process or procedure.
3) In case of C, the data is not secured	3) In C++, the data is secured i.e. the data Is hidden
4) C uses the top-down approach	4) C++ uses the bottom-up approach
5) C does not supports for function overloading	5) C++ supports for function overloading
6) We cannot use functions inside structures in C	6) We can use functions inside structures in C++
7) ‘C’ uses scanf( ) & printf ( ) for input & output functions respectively.	7) C++ uses ‘cin’ & ‘cout’ objects for input and output operations respectively.
8) ‘C’ does not allows the use of reference variables.	8) C++ allows the use of reference variables.
9) Reusability of code is not possible	9) Reusability of code is possible
10) C program has extension .C	10) C++ program has extension .cpp
11) It is function driven language	11) It is object driven language.

Assignment: 1

1) What is Programming language? Define procedure oriented and object oriented Programming language.
2) What is POP? List out its characteristics.
3) What is OOP? List out its characteristics.
4) Differentiate between POP and OOP.
5) Explain following OOP’s concepts:
   1) Class 2) Object 3) Data Encapsulation 4) Data abstraction
   5) Polymorphism 6) Inheritance 7) Message passing
6) Differentiate between ‘C’ and ‘C++’ language
7) Differentiate between ‘Early Binding’ and ‘Late binding’.
9) Differentiate between ‘Class’ and ‘Structure’
10) What is ‘C++’? Write its history and application.
11) Explain ‘cin’ and ‘cout’ objects.
12) What is data type? Explain primitive data types in ‘C++’